

Bataille Navale

SOMMAIRE

Etape 1 : initialisation des tableaux tabjoueur & tabordi.....	3
Etape 2 : Placement des 5 pions : tabjoueur.....	4
Etape 3 : Placement des 5 pions de l'ordinateurs : tabordi.....	7
Etape 4 : créer la procédure affichagetab.....	9
Etape 5 : créer la procédure affichagetabCache.....	11
Etape 6 : Le joueur cherche un pion.....	12
Etape 7 : L'ordinateur cherche un pion.....	15
Etape 8 : fin de partie ?.....	17

Etape 1 : initialisation des tableaux tabjoueur & tabordi

8.1 Navigation

Pour cet exercice, nous allons déclarer un tableau de 5 lignes et 5 colonnes. ↴

```
import java.util.Scanner;

public class bataillenavale {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // étape 1 : déclaration du tableau

        int tabjoueur[][]= new int [5][5];
        int tabordi[][]= new int [5][5];
```



Nous allons parcourir chaque ligne et chaque colonne du tableau afin d'y placer la valeur 0.

Cela permet d'avoir un plateau entièrement vide avant de commencer la partie.

↴

```
import java.util.Scanner;

public class bataillenavale {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // étape 1 : déclaration du tableau

        int tabjoueur[][]= new int [5][5];
        int tabordi[][]= new int [5][5];

        for (int i = 0; i < 5; i++) {
            for (int j = 0; j < 5; j++) {
                tabjoueur[i][j] = 0;
                tabordi[i][j] = 0;
            }
        }

    }

}
```



Explication :

```
for (int i = 0; i < 5; i++) { ↵
```

Nous allons créer une variable *i* qui représente le numéro de ligne, et nous allons passer de la ligne 0 à la ligne 4.

```
    for (int j = 0; j < 5; j++) { ↵
```

Pour chaque ligne *i*, nous allons créer une variable *j* qui représente la colonne, et nous allons passer de la colonne 0 à la colonne 4.

```
        tabjoueur[i][j] = 0; ↵
```

Nous allons mettre la valeur 0 dans la case du tableau du joueur située à la ligne *i* et la colonne *j*.

```
        tabordi[i][j] = 0; ↵
```

Nous allons aussi mettre la valeur 0 dans la même case (ligne *i*, colonne *j*) du tableau de l'ordinateur.

Etape 2 : on place nos 5 pions : tabjoueur

2.1 Nous allons demander à l'utilisateur à quelle ligne il veut placer son pion. ↵

```
import java.util.Scanner;

public class bataillenavale {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // étape 1 : déclaration du tableau

        int tabjoueur[][]= new int [5][5];
        int tabordi[][]= new int [5][5];

        for (int i = 0; i < 5; i++) {
            for (int j = 0; j < 5; j++) {
                tabjoueur[i][j] = 0;
                tabordi[i][j] = 0;
            }
        }

        System.out.println("A quelle ligne souhaitez-vous placer votre pion ? (Entre 1 et 5");
        int ligne = sc.nextInt();

    }
}
```



2.2 Nous faisons pareil pour les colonnes.

```
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        tabjoueur[i][j] = 0;
        tabordi[i][j] = 0;
    }
}

System.out.println("A quelle ligne souhaitez-vous placer votre pion ? (Entre 1 et 5");
int ligne = sc.nextInt();

System.out.println("A quelle colonne souhaitez-vous placer votre pion ? (Entre 1 et 5");
int colonne = sc.nextInt();
```



Par ailleurs l'utilisateur ne doit placer un pion si l'endroit est déjà pris et si l'endroit est hors du périmètre du tableau.

Dans notre programme, nous avons utilisé des boucles **while** pour contrôler la saisie de l'utilisateur.

Nous avons choisi ce type de boucle car elle permet de **répéter une question tant que la valeur entrée n'est pas correcte.** ↴

```
// Saisie pion ligne
while (ligne < 1 || ligne > 5) {
    System.out.println("A quelle ligne souhaitez-vous placer votre pion ? (Entre 1 et 5)");
    ligne = sc.nextInt();

    if (ligne < 1 || ligne > 5) {
        System.out.println("Attention, veuillez placer votre pion entre 1 et 5.");
    }
}

// Saisie pion colonne
while (colonne < 1 || colonne > 5) {
    System.out.println("A quelle colonne souhaitez-vous placer votre pion ? (Entre 1 et 5)");
    colonne = sc.nextInt();

    if (colonne < 1 || colonne > 5) {
        System.out.println("Attention, veuillez placer votre pion entre 1 et 5.");
    }
}
```

2.3 Lorsque le joueur choisit une ligne et une colonne valides, nous allons vérifier que la case n'est pas déjà occupée.

Si tout est correct, nous allons alors mettre la valeur **1** dans cette case du tableau. Cela signifie que nous venons de poser un pion à cet endroit. ↴

```
// Verifier si la case est déjà occupée
if (tabJoueur[ligne - 1][colonne - 1] == 1) {
    System.out.println("Attention, cette case contient déjà un pion.");
} else {
    tabJoueur[ligne - 1][colonne - 1] = 1;
    System.out.println("Pion placé");
}
```



2.4 Nous allons créer une variable **nbPions** qui sert à compter le nombre de pions déjà placés par le joueur.

Au début, nous mettons cette valeur à 0, car aucun pion n'est encore posé. ↴

```
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        tabJoueur[i][j] = 0;
        tabordi[i][j] = 0;
    }
}

int nbPions = 0; // Compteur de pions
while (nbPions < 5) {
```



Et nous allons utiliser une boucle **while** pour répéter les étapes de saisie et de placement tant que le joueur n'a pas placé ses 5 pions.

```
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        tabJoueur[i][j] = 0;
        tabordi[i][j] = 0;
    }
}

int nbPions = 0; // Compteur de pions
while (nbPions < 5) {
```



Résultat ↴

```
A quelle ligne souhaitez-vous placer votre pion ? (Entre 1 et 5)
4
A quelle colonne souhaitez-vous placer votre pion ? (Entre 1 et 5)
3
Pion placé (2/5)
A quelle ligne souhaitez-vous placer votre pion ? (Entre 1 et 5)
```

Etape 3 : on place les 5 pions de l'ordinateurs : tabordi

3.1 Nous allons attribuer une ligne aléatoirement de l'ordinateur entre 1 et 5 :

```
    } else {
        tabJoueur[ligne - 1][colonne - 1] = 1;
        nbPions++;
        System.out.println("Pion placé (" + nbPions + "/5)");
    }
}

// Etape 3, 3.1 :
int ligneOrdi = (int)(Math.random() * 5) + 1;
System.out.println("L'ordinateur a choisi la ligne : " + ligneOrdi);
```



Note : Nous avons utilisé la fonction **Math.random()** afin de générer un nombre aléatoire compris entre 1 et 5 et nous plaçons ensuite ce nombre dans la variable **ligneOrdi**.

Cela nous permet d'attribuer automatiquement une ligne au pion de l'ordinateur.

3.2 Nous faisons pareil pour les colonnes. ↴

```
// Etape 3, 3.1 :
int ligneOrdi = (int)(Math.random() * 5) + 1;
System.out.println("L'ordinateur a choisi la ligne : " + ligneOrdi);

// 3.2
int colonneOrdi = (int)(Math.random() * 5) + 1;
System.out.println("L'ordinateur a choisi la colonne : " + colonneOrdi);
```



Nous allons faire en sorte que l'ordinateur ne place pas un pion sur une case déjà occupée.

```
int colonneOrdi = (int)(Math.random() * 5) + 1;
System.out.println("L'ordinateur a choisi la colonne : " + colonneOrdi);

if (tabordi[ligneOrdi - 1][colonneOrdi - 1] == 1) {
} else {
    tabordi[ligneOrdi - 1][colonneOrdi - 1] = 1;
}
```



3.3 Pour cette étape, nous devons vérifier si la case choisie par l'ordinateur est libre.

Si cette case contient la valeur 0, cela signifie que le choix est valable.

Nous pouvons alors y placer un pion en mettant la valeur 1 dans le tableau. ↴

```
// 3.3 : Tableau ordinateur valeur 1

if (tabordi[ligneOrdi - 1][colonneOrdi - 1] == 0) {
    tabordi[ligneOrdi - 1][colonneOrdi - 1] = 1;
}
```

3.4 Nous allons faire en sorte que l'ordinateur place 5 pions sur son propre tableau, comme nous l'avons fait auparavant pour le joueur. ↴

```
if (tabordi[ligneOrdi - 1][colonneOrdi - 1] == 0) {
    tabordi[ligneOrdi - 1][colonneOrdi - 1] = 1;
    nbPionsOrdi++;
    System.out.println("Pion de l'ordinateur placé(" + nbPionsOrdi + "/5)");
} else {
    System.out.println("La case est déjà occupée, l'ordinateur recommence.");
}
}
```


Résultat

```
L'ordinateur a choisi la colonne : 3
Pion de l'ordinateur placé(4/5)
L'ordinateur a choisi la ligne : 2
L'ordinateur a choisi la colonne : 3
La case est déjà occupée, l'ordinateur recommence.
L'ordinateur a choisi la ligne : 1
L'ordinateur a choisi la colonne : 3
Pion de l'ordinateur placé(5/5)
```

Etape 4 : créer la procédure affichagetab

4.1 Dans l'exercice 4.1, nous allons créer la procédure `affichagetabJoueur`. Cette procédure recevra deux paramètres : le tableau du joueur et le nombre de cases. ↴

```
}

static void affichagetabJoueur(int tab[][], int nbcase) {

}
```

4.2 Dans cette étape, nous allons commencer à créer l'affichage du tableau du joueur. Afficher les espaces et les 5 premiers numéros (1,2,3,4,5)

-> Nous affichons deux espaces afin de créer le coin supérieur gauche du tableau.

-> Nous utilisons une boucle `for` qui va de 1 jusqu'à `nbcase`.

```
for (int i = 1; i <= nbcase; i++)
    System.out.print(i + " "); ↴
```

```
// 4.2 : tableau joueur
System.out.print(" ");

for (int i = 1; i <= nbcase; i++) {
    System.out.print(i + " ");
}

System.out.println();
}
```

Affichage du tableau

```
        System.out.println("Pion de l'ordinateur placé("
    } else {
        System.out.println("La case est déjà occupée, l'
    }
}

affichageTabJoueur(tabJoueur, 5);
```

4.3 et 4.4 Nous allons compléter notre procédure `affichageTabJoueur` pour qu'elle affiche tout le tableau du joueur, ligne par ligne.

À l'intérieur d'une boucle qui parcourt toutes les lignes du tableau, nous commençons par afficher ↴

```
// numéro 1,2,3,4 et 5
System.out.print((i + 1) + " ");
```

Cela nous permet d'afficher les numéros 1, 2, 3, 4 et 5 sur la gauche du tableau, comme dans un vrai plateau.

Ensuite pour chaque ligne, nous utilisons une seconde boucle, grâce à cette boucle, nous vérifions le contenu de chaque case du tableau.

```
for (int j = 0; j < nbcase; j++) {
```

Dans chaque case :

-> si la valeur vaut 1, cela signifie qu'il y a un pion, donc nous affichons "o"

-> si la valeur vaut 0, il n'y a pas de pion, donc nous affichons "~"

```
    for (int j = 0; j < nbcase; j++) {
        if (tab[i][j] == 1) {
            System.out.print("o ");
        } else {
            System.out.print("~ ");
        }
    }
}
```

4.5 Affichage du tableau

```
L'ordinateur a choisi la ligne : 3
L'ordinateur a choisi la colonne : 1
Pion de l'ordinateur placé(5/5)
 1 2 3 4 5
1 0 ~ ~ ~ ~
2 ~ ~ ~ ~ 0
3 0 ~ ~ ~ 0
4 ~ 0 ~ ~ ~
5 ~ ~ ~ ~ ~
```



Etape 5 : créer la procédure affichagetabCache

Dans l'étape 5 nous souhaitons créer une nouvelle procédure appelée

affichagetabCache.

Cette procédure va nous permettre d'afficher le plateau caché, c'est-à-dire ce que le joueur doit voir quand il cherche un pion ennemi.

Comme pour l'étape 4, nous affichons d'abord les numéros de colonnes

```
// Etape 5
System.out.print(" ");
for (int i = 1; i <= nbcase; i++) {
    System.out.print(i + " ");
}
System.out.println();
```

Ensuite nous parcourons toutes les lignes du tableau

```
// Etape 5
System.out.print(" ");
for (int i = 1; i <= nbcase; i++) {
    System.out.print(i + " ");
}
System.out.println();

for (int i = 0; i < nbcase; i++) {
```



Pour les cases non découvertes non affichons un “?”

```
// Case non découverte
if (tab[i][j] == 0 || tab[i][j] == 1) {
    System.out.print("? ");
}
```

Pion découvert

```
System.out.print("? ");
}
// Pion découvert
else if (tab[i][j] == 2) {
    System.out.print("o ");
}
```

Case découverte sans pion

```
// Case découverte mais sans pion
else if (tab[i][j] == 3) {
    System.out.print("x ");
}
```

Etape 6 : Le joueur cherche un pion

6.0 Nous affichons à l'utilisateur qu'il peut commencer à jouer

```
// 6.0 : Afficher "A vous de jouer !"
System.out.println("A vous de Jouer !");
}
```

6.1 Nous demandons l'utilisateur à quelle ligne il veut découvrir une case

```
// 6.1 : Demandez à l'utilisateur à quelle ligne il veut découvrir une case
int ligneChoisie = 0;
while (ligneChoisie < 1 || ligneChoisie > 5) {
    System.out.println("À quelle ligne souhaitez-vous découvrir une case ? (Entre 1 et 5)");
    ligneChoisie = sc.nextInt();

    if (ligneChoisie < 1 || ligneChoisie > 5) {
        System.out.println("Attention, veuillez choisir une ligne entre 1 et 5.");
    }
}
```

6.2 Nous faisons pareil mais pour les colonnes.

```
// 6.2 : Demandez à l'utilisateur à quelle colonne il veut découvrir une case
int colonneChoisie = 0;
while (colonneChoisie < 1 || colonneChoisie > 5) {
    System.out.println("À quelle colonne voulez-vous découvrir une case ? (Entre 1 et 5)");
    colonneChoisie = sc.nextInt();

    if (colonneChoisie < 1 || colonneChoisie > 5) {
        System.out.println("Attention, veuillez choisir une colonne entre 1 et 5.");
    }
}
```

6.3 Dans cet exercice, nous allons mettre un timer de 2 secondes pour simuler le tir.

- > **Thread.sleep(2000)** Nous demandons au programme de s'arrêter pendant 2000 millisecondes, c'est-à-dire 2 secondes.

```
// 6.3 : timer 2 secondes
System.out.println("Tir en cours");
try {
    Thread.sleep(2000);
} catch (InterruptedException e) {
}
```

6.4 Affichage du résultat

```
// 6.4 : Affichage du résultat
if (tabordi[ligneChoisie - 1][colonneChoisie - 1] == 0) {
    System.out.println("Raté");
    tabordi[ligneChoisie - 1][colonneChoisie - 1] = 3;
} else if (tabordi[ligneChoisie - 1][colonneChoisie - 1] == 1) {
    System.out.println("Touché");
    tabordi[ligneChoisie - 1][colonneChoisie - 1] = 2;
    nbPionTrouveJoueur++;
} else {
    System.out.println("Case déjà découverte");
}
```

if (tabordi[ligneChoisie - 1][colonneChoisie - 1] == 0 Nous testons ici si la case que le joueur a visée dans le tableau de l'ordinateur contient 0

,System.out.println("Raté") : Si la case ne contient pas de pion, nous affichons « Raté » pour signaler que le tir n'a touché aucune cible.

tabordi[ligneChoisie - 1][colonneChoisie - 1] = 3 Nous remplaçons la valeur de cette case par 3 pour indiquer qu'elle a été découverte et qu'il n'y avait pas de pion à cet endroit.

else if (tabordi[ligneChoisie - 1][colonneChoisie - 1] == 1 Si la case contient la valeur 1, cela signifie qu'un pion de l'ordinateur s'y trouvait.

System.out.println("Touché !") Affichage que le pion adverse a été touché

tabordi[ligneChoisie - 1][colonneChoisie - 1] = 2 Nous remplaçons la valeur de la case par 2 pour indiquer qu'un pion a été découvert à cet endroit.

nbPionTrouveJoueur++ Nous augmentons de 1 la variable nbPionTrouveJoueur, qui compte le nombre de pions ennemis que le joueur a trouvés.

Else Si la case n'est ni 0 ni 1, cela veut dire qu'elle contient déjà une valeur 2 ou 3. Dans ce cas, cela signifie que nous avons déjà tiré sur cette case auparavant.

6.5 Nous allons afficher le tableau de l'ordinateur

```
// 6.5 : Affichage tableau ordinateur  
affichagetabCache(tabordi, 5);  
}
```

Résultat :

```
Tir en cours...  
Raté !  
  1 2 3 4 5  
1 ? ? ? ? ?  
2 ? ? ? ? ?  
3 ? ? ? ? ?  
4 ? ? ? ? ?  
5 ? ? x ? ?
```

Etape 7 : L'ordinateur cherche un pion

7.0 Affichage "tour de l'ordinateur" ↴

```
// 7.0 : Affichage "tour de l'ordinateur"  
System.out.println("Tour de l'ordinateur");  
}
```

7.1 Nous déclarons une variable entière appelée **ligneTirOrdi** qui va contenir le numéro de ligne choisi par l'ordinateur.

Ensuite, nous utilisons la fonction **Math.random()** pour générer un nombre aléatoire. ↴

```
// 7.1 : Attribution ligne aléatoire  
int ligneTirOrdi = (int)(Math.random() * 5) + 1;  
System.out.println("L'ordinateur choisit la ligne : " + ligneTirOrdi);  
}
```

7.2 Attribution d'une colonne aléatoirement

```
// 7.2 : Attribution colonne aléatoire
int colonneTirOrdi = (int)(Math.random() * 5) + 1;
System.out.println("L'ordinateur choisit la colonne : " + colonneTirOrdi);
}
```

7.3 Nous utilisons une boucle **while** pour vérifier que la case choisie par l'ordinateur est bien valide.

Nous accédons à la case du tableau du joueur en utilisant les coordonnées choisies par l'ordinateur :

- **ligneTirOrdi - 1** → car le tableau commence à l'indice 0,
- **colonneTirOrdi - 1** → pour la même raison aussi

```
ligneTirOrdi = (int)(Math.random() * 5) + 1;
```

Nous tirons une nouvelle ligne aléatoire entre 1 et 5.

```
colonneTirOrdi = (int)(Math.random() * 5) + 1;
```

Nous tirons aussi une nouvelle colonne aléatoire entre 1 et 5.

```
// 7.3 : Verification case non découverte par l'ordinateur
while (tabjoueur[ligneTirOrdi - 1][colonneTirOrdi - 1] != 0 && tabjoueur[ligneTirOrdi - 1][colonneTirOrdi - 1] != 1) {
    ligneTirOrdi = (int)(Math.random() * 5) + 1;
    colonneTirOrdi = (int)(Math.random() * 5) + 1;
}
```

7.4 Nous affichons le message <<tir en cours>> pour signaler que l'ordinateur est en train d'effectuer son tir. ↴

```
System.out.println("Tir en cours");
try {
    Thread.sleep(2000);
} catch (InterruptedException e) {
}
}
```


7.5 Affichage du résultat

```
// 7.5 : Affichage du résultat
if (tabJoueur[ligneTirOrdi - 1][colonneTirOrdi - 1] == 0) {
    System.out.println("Raté");
    tabJoueur[ligneTirOrdi - 1][colonneTirOrdi - 1] = 3;
} else if (tabJoueur[ligneTirOrdi - 1][colonneTirOrdi - 1] == 1) {
    System.out.println("Touché");
    tabJoueur[ligneTirOrdi - 1][colonneTirOrdi - 1] = 2;
    nbPionTrouveOrdi++;
}
}
```

`if (tabJoueur[ligneTirOrdi - 1][colonneTirOrdi - 1] == 0)` Nous commençons par vérifier ce que contient la case du joueur que l'ordinateur a visée :

`System.out.println("Raté !");`

`tabJoueur[ligneTirOrdi - 1][colonneTirOrdi - 1] = 3;` Nous affichons « Raté » et nous mettons la valeur 3 dans le tableau pour indiquer que la case a été découverte sans pion.

`else if (tabJoueur[ligneTirOrdi - 1][colonneTirOrdi - 1] == 1)` Si la case contient 1, cela veut dire qu'il y avait un pion du joueur.

`System.out.println("Touché !");`

`tabJoueur[ligneTirOrdi - 1][colonneTirOrdi - 1] = 2;`

`nbPionTrouveOrdi++;` Nous affichons « Touché »

Et nous augmentons le compteur `nbPionTrouveOrdi` pour suivre le nombre de pions trouvés.

7.6 Affichage du tableau du joueur ↴

```
// 7.6 : Affichage tableau joueur
affichageTabCache(tabJoueur, 5);
}
```

Etape 8 : fin de partie ?

8.1 Nous souhaitons à présent que la partie se répète automatiquement tant qu'aucun des deux joueurs n'a gagné.

Pour cela, nous utilisons une boucle **while**. Nous plaçons la ligne avant l'étape 6 avant le joueur commence à jouer. ↴

```
// 8.1 : répétition des étapes 6 et 7
while (nbPionTrouveJoueur < 5 && nbPionTrouveOrdi < 5) {
```

8.2 Nous affichons le vainqueur ↴

```
// 8.2 : Affichage du vainqueur
if (nbPionTrouveJoueur == 5) {
    System.out.println("Félicitations vous avez gagné!");
} else {
    System.out.println("Malheureusement l'ordinateur à gagné");
}
```

8.3 Pour finir nous allons demander à l'utilisateur s'il souhaite refaire une partie ↴

```
// 8.3 : Affichage refaire partie
System.out.println("Souhaitez-vous refaire une partie ? (o/n)");
char reponse = sc.next().charAt(0);
if (reponse == 'o' || reponse == 'O') {
    main(args);
} else {
    System.out.println("Fin du jeu !");
}
}
```

Résultat :

```
1 x x x x x
2 o x o x x
3 x x x x x
4 o x x x x
5 x x ? o x
Félicitations, vous avez gagné !
Souhaitez-vous refaire une partie ? (o/n)
n
Fin du jeu !
```